Query Processing

Abu Raihan Mostofa Kamal

Professor, CSE Department
Islamic University of Technology (IUT)
Gazipur-1704 Bangladesh
raihan.kamal@iut-dhaka.edu
+8801843925543

March 9, 2026

This is a sample footnote just to show how it works

Chapter Outline

Overview of Basic Steps

Query Cost

Cost of Selection Operations

Introduction

- Query processing refers to the range of activities involved in extracting data from a database.
- It involves translation of queries, a variety of query-optimizing transformations, and actual evaluation of queries.

Process Overview

The steps involved in processing a query involves are:

1. Parsing and translation.
2. Optimization.
3. Evaluation.

Process Overview: Parsing and translation

The steps involved in processing a query involves are:

1. It translates a given query into its internal form. This translation process is similar to the work performed by the parser of a compiler.
2. In generating the internal form of the query, the parser checks the syntax of the users query
3. It veries that the relation names appearing in the query are names of the relations in the database, and so on
4. Finally, the system constructs a parse-tree representation of the query, which it then translates into a relational-algebra expression.

Parsing and translation: Postgre SQL example

- PostgreSQL parser consists of source files primarily located in **`src/backend/parser`** within the source code distribution.
- These files define the SQL grammar and lexical analysis rules, which are used to convert a plain text query into a machine-readable internal parse tree or Abstract Syntax Tree (AST).
- Standard Unix tools are used for the core functions.

Parsing and translation: Postgre SQL example (Cont.)

Key Parser Files and Tools:

- **`scan.l`** (Lexer definition file): Uses flex to perform lexical analysis, recognizing keywords, identifiers, and literals, generating tokens for the parser.
- **`gram.y`** (Grammar definition file): The file uses Backus-Naur Form (BNF) or a similar notation to describe the structure of the language. **Example:** A rule might state that a **`SELECT_STATEMENT`** must consist of the token SELECT, followed by a **`COLUMN_LIST`**, then the token FROM, and finally a **`TABLE_NAME`** It also builds an **Abstract Syntax Tree (AST)**a

  hierarchical representation of the query that the database can actually execute.
- **`parsenodes.h`** (Header file): Defines the data structures (C structs) used to represent the nodes within the parse tree.
- **`analyze.c, parse_*.c`** files: These files contain the "analyzer" or "transformer" functions, which add semantic information to the raw parse tree by looking up system catalogs (e.g., verifying table and column existence)

Lexer Definition File: Postgre SQL example (Cont.2)

```
/* Definitions */
space       [ \t\n\r\f]
identifier  [A-Za-z][A-Za-z0-9_]*
digit       [0-9]
integer     {digit}+
-----
"--".*      { /* ignore SQL comments */ }

"/*"        { /* Handle nested C-style comments (simplified) */
  /* In real Postgres, this calls a complex function */
}
```

Basic Steps: Optimization

This is arguably the most critical step.

- **Cost Estimation:** The optimizer looks at "metadata" (statistics about the data) like how many rows are in a table or if an index exists.
  SQL, a query could be expressed in **several different ways**. Each SQL query can itself be translated into a relational-algebra expression in one of several ways.
- **Plan Generation:** It evaluates different execution plans. For example, should it use an index to find a specific record, or just scan the whole table? Should it Join Table X to Y, or Y to X?
- **Selection:** The goal is to find the plan with the lowest "cost," usually measured in terms of I/O operations and CPU time.

Basic Steps: Optimization (Cont.)

Example:

```
select salary
from instructor
where salary < 50000;
```

Its relational algebra expression may have multiple equivalent expressions:

$$\delta_{salary<50000}(\pi_{salary}(instructor)) \equiv \pi_{salary}(\delta_{salary<50000}(instructor))$$

- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan** or **query-execution plan**.
- Example:
  - ✓ **Option 1:** Use an index on salary to find instructors with $salary < 75000$
  - ✓ **Option 2:** Perform complete relation scan and discard instructors with $salary > 75000$

Basic Steps: Evaluation

Once the optimizer picks the best plan, followings are done:

- **Query Execution Engine**: This engine takes the chosen execution plan and runs it against the actual database files.
- **Physical Operators**: It uses specific algorithms to get the job done, such as Hash Joins, Index Scans, or External Sort-Merge.
- **Query Result**: As the data is processed, the engine sends the results back to the user.

Overall Picture

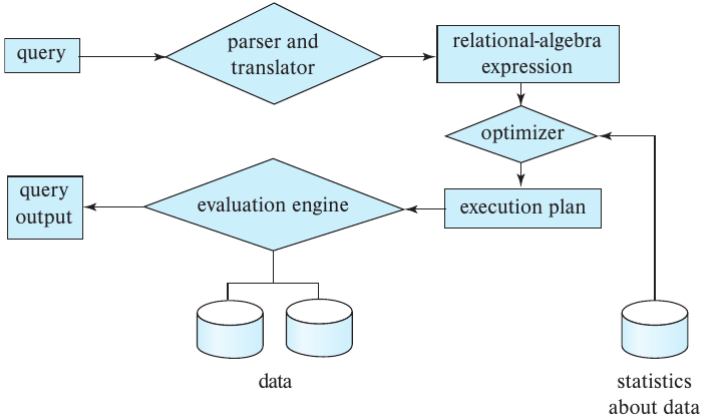

Figure: Overview of Query Processing

Measures of Query Cost

- Many factors contribute to time cost
  Example *disk access, CPU, and network communication*
- Cost can be measured based on:
  - ✓ **Response time**, i.e. total elapsed time for answering query,
    or
  - ✓ Total **resource consumption**
- We use total **resource consumption as cost metric**.
  Since: Response time is **harder to estimate**, and minimizing resource consumption is a good idea in a shared database. Response time depends on the particular type of hardware often, and type of hardware (ssd vs. hdd)
- We ignore CPU costs for simplicity:
  - ✓ Real systems do take CPU cost into account
  - ✓ Network costs must be considered for parallel systems
- We also do not include cost to writing output to disk (for simplicity)

Measures of Query Cost (Cont.)

- Disk cost can be estimated as:
  - ✓ Number of seeks × average-seek-cost
  - ✓ Number of blocks read × average-block-read-cost
  - ✓ Number of blocks written × average-block-write-cost
- For simplicity we just use the number of block transfers from disk and the number of seeks as the cost measures.
  - ✓ t_T time to transfer one block
    (Assuming for simplicity that write cost is same as read cost)
  - ✓ t_S time for one seek
  - ✓ Cost for b block transfers plus S seeks
    $b * t_T + S * t_S$
- Typical values of t_T and t_S (assuming 4KB blocks)
  - ✓ **High end magnetic disk:** t_S = 4 msec and t_T =0.1 msec
  - ✓ **SSD:** t_S = 20-90 microsec and t_T = 2-10 microsec for 4KB

Measures of Query Cost (Cont.)

Why is Response Time hard to estimate?

Apart from the type of storage used followings are some important considerations:

- **Effective Main Memory:** The costs of all the algorithms depend on the **size of the buffer in main memory**. In the **best case**, if data ts in the buffer, the data can be read into the buffers, and the disk does not need to be accessed again.
  In the **worst case**, we may assume that the buffer can hold only a few blocks of data approximately one block per relation.
  However, with large main memories available today, such worst-case assumptions are overly pessimistic.
  Example: PostgreSQL the total memory available to a query, called the effective cache size, is assumed by default to be **4 gigabytes**, if a query has several operators that run concurrently, the available memory has to be divided amongst the operators.

Measures of Query Cost (Cont.)

Why is Response Time hard to estimate? (Cont.)

- **Caching in Main Memory:** We assume that data must be read from disk initially, but it is possible that a block that is accessed is **already present in the in-memory buffer** i.e main memory.
  - ✓ **Example:** PostgreSQL uses the following trick: the cost of a random page read is assumed to be 1/10th of the actual random page read cost, to model the situation that 90% of reads are found to be resident in cache.
  - ✓ most database systems assume that all internal nodes are present in the in-memory buer, and assume that a traversal of an index only incurs a single random I/O cost for the leaf node.
- **Multiple Disk Access:** In a system with multiple disks, the response time depends on how accesses are distributed among disks, which is hard to estimate without detailed knowledge of data layout on disk.

The stated factors must be accumulated in computing Response Time which is very dynamic. Hence estimating Response Time is difficult.

Selection Operation: Background

File scan

In query processing, the file scan is the lowest-level operator to access data. File scans are search algorithms that locate and retrieve records that fulfill a selection condition. In relational systems, a file scan allows **an entire relation to be read** in those cases where the relation is **stored in a single, dedicated file**.

Cost Function for SQL operation

Cost functions are primarily based on the number of **block transfers** and **disk seeks** required to **find the matching records**.

Selection Operation: Background

- **Clustered (Clustering) Index:** A clustered index determines the physical order of records in the data file.
  Key points
  - ✓ Records are stored on disk in the same order as the index key
  - ✓ Records with the same key value are stored together
  - ✓ Usually only one clustered index per table
  - ✓ Very fast for range queries and needs fewer disk seeks
- **Non-Clustered (Secondary) Index:** A non-clustered index does NOT affect the physical order of records.
  - ✓ Index stores pointers to records **stored elsewhere**
  - ✓ Records with the same key value are **scattered**
  - ✓ Multiple non-clustered indices are allowed

Selection Operation: A1 Linear Search

- A1 : It scans every block of the relation and test every record to see if it satisfies the selection condition.

  Linear search, regardless of availability of indices

  ✓ No index needed
  Recall: Because primary key is a logical constraint, while index is a physical structure. They are related, but not the same thing.
  (A physical data structure (B+ tree, hash, etc.) Exists only if the DB creates it Used to speed up searches)

  ✓ Works for any condition (=, <, >, LIKE, etc.)

  ✓ Works even if the file is unsorted

  ✓ This is the most basic way to evaluate a selection **Seq Scan / Table Scan**

A1 Linear Search: 2 cases

Example:

```
select address
from emp
where dept='CSE';
```

where **dept** is **not a key** of the relation.

i. **Non Key Selection** Here the selection may return 0 or more records, in other words, the search must continue till the last block.
So, assuming the table records spans over b blocks

Cost: $t_S + b_r * t_T$ (initial seek time to locate the object plus searching all the blocks)

Since **dept is not a key of the relation there is no way to estimate the number of records to be returned, so search must continue till the end. Hence b_r blocks are needed**

A1 Linear Search: 2 cases (Cont.)

Example:

```
select address
from emp
where empID=105;
```

where `empID` is **a key** of the relation.

ii. **Keyed Selection:** Here the number of records to be returned is **maximum of 1** (as key in where clause) **But:** this case only assumes key in the logical design not physical index (i.e. data path in B+ tree for instance does not exist). **Cost:** $t_S + (b_r)/2 * t_T$

Best case: 1 search (first one in the where clause mentioned)
Worst case: Keep searching till the end: b_r searches needed
So, **average** has been taken.

A2: Clustering Index (Primary Key), Equality on Key

- Algorithm A2 is used when a selection condition is an equality on a primary key and a clustering index exists on that key. Since the search is on a key, **only a single record will satisfy the condition**.
- SQL Example:

```sql
SELECT *
FROM Student
WHERE sid = 101;
```

Where `sid` is the primary key and **database creates a clustering B+ tree index on `sid`**

- **Cost Function:** $\text{Cost} = (h_i + 1) * (t_T + t_S)$ where
  - ✓ h_i: The height of the index (number of levels to traverse).
  - ✓ t_T: Time to transfer one block.
  - ✓ t_S: Time for one seek.
  - ✓ The $+1$: Represents the final block transfer and seek to retrieve the actual data record after finding its pointer in the index.

A3: Clustering Index, Equality on Non-key

- Algorithm A3 is used for an equality condition on a non-key attribute that has a clustering index. Because it is a non-key, multiple records might satisfy the condition. Since the index is clustering, these records will be stored on consecutive blocks.
- SQL Example:

```
SELECT *
FROM Student
WHERE dept_name = 'CSE';
```

✓Where *dept_name* is not unique
✓There is a clustering index on *dept_name*
✓Many students belong to "CSE"

- **Cost Function:** Cost $= h_i * (t_T + t_S) + t_S + t_T * b$
  where
  - ✓ $h_i * (t_T + t_S)$: Cost to traverse the index to the first matching record.
  - ✓ t_S: One seek to reach the first data block.
  - ✓ $t_T * b$: The time to transfer b blocks, where b is the total number of consecutive blocks containing matching records(i.e. all students of CSE dept).

A3: Clustering Index, Equality on Non-key (Cont.)

- B+ tree on a non-key attribute like *dept_name*, the search-key values can indeed be **repeated**. This is because multiple instructors can belong to the same department, meaning several records will share the same *dept_name* value.
- When dealing with such repeated values, the system distinguishes between whether the index is clustering or secondary to determine how to retrieve the data efficiently (choose between A3 or A4 (next one))

Algorithm A4: Secondary Index, Equality Search

- The efficiency and cost of A4 depend on whether the search attribute is a candidate key (unique) or a non-key (repeated). So, it has 2 cases.
- **Case 1:** If the search-key is a candidate key, only a single record can satisfy the condition.
  Cost:$= (h_i + 1) * (t_T + t_S)$
  ✓h_i: Height of the B+ tree index, t_T: Time for one block transfer. And t_S: Time for one disk seek
  ✓Note for +1: he system traverses the height of the index to find the pointer and then **performs one additional block transfer** to fetch the actual record.

```
SELECT *
FROM Student
WHERE sid = 101;
```

Algorithm A4: Secondary Index, Equality Search

- **Case 2:** Equality on a Non-key (repeated value possible, should be expensive)
  Multiple (say n) records may satisfy the condition.
  Cost: $= (h_i + n) * (t_T + t_S)$
  ✓**Explanation:** Since the index is secondary, matching records are likely scattered across different disk blocks. In the worst case, the system must perform a separate disk seek and block transfer for each of the n matching records.
  ✓This can be **very expensive**, sometimes even more so than a linear file scan (A1) if n is very large.
  ✓h_i:

```
-- Assuming dept_name has a secondary index and multiple instructors
   belong to 'CSE'
SELECT * FROM instructor
WHERE dept_name = 'CSE';
```